Science

Cryptography: Extended Euclidean Algorithm

Topics

- Extended Euclidean Algorithm ←
- Modular Arithmetic
- Diffie-Hellman Key Exchange
- Public Key Cryptography

Euclidean Algorithm

The greatest common divisor $gcd(a, b)$ of two natural numbers a and b is the greatest number that divides both a and b. This number plays an important role in the number theory and public cryptography.

There is conceptually a simple way to find the gcd. Factor both numbers a and b into a product of primes and multiply together all common factors. The problem with this approach is that it is computationally extremely difficult to factor large numbers into primes.

Already in the 3rd century BC, the greek mathematician Euclid described an ingenious and very efficient algorithm to compute the gcd. The algorithm is based on the following fact (assume $a \geq b > 0$):

$$a = q \cdot b + r; \quad 0 \leq r < b \tag{1}$$

Here q is the quotient, i.e. the result of the integer division $a \,//\, b$, and r is the remainder $r = a \,\%\, b = a \bmod b$. Now we have two cases:

1. Either $r = 0$. Then b divides a and therefore we found $gcd(a, b) = b$.
2. Or $r > 0$. Because $r = a - q \cdot b$ all divisors of a, b are also divisors of b, r and vice versa. So,

   $$gcd(a, b) = gcd(b, r) \quad \text{and} \quad b < a, r < b \tag{2}$$

   and we have reduced the problem to the same problem but with smaller numbers. We can now repeat this process with smaller numbers such that it must finally terminate:

   $$a' = q' \cdot b' + r' \quad \text{and} \quad r' = 0, b' = gcd(a, b) \tag{3}$$

The described process can immediately be cast into a recursive algorithm in Python:

```
def gcd_rec(a, b):
    if b == 0:
        return a
    return gcd_rec(b, a % b)
```

An iterative algorithm is also straightforward (Python):

```
def gcd(a, b):
    while b:
        q = a // b
```

```
        a, b = b, a % b
    return a
```

The last assignment in the above program is a parallel (vectorial) assignment that is possible in Python.

Bézout's Identity

The $gcd(a, b)$ has an interesting and very useful property: it is a linear combination of a and b with integer coefficients:

$$gcd(a, b) = a \cdot x + b \cdot y \qquad (x \text{ and } y \text{ integers}) \tag{4}$$

This relation is called the Bézout's identity. We will prove it constructively and at the same time present the Extended Euclidean Algorithm to compute the coefficients x and y in equation (4).

Extended Euclidean Algorithm

The Euclidean Algorithm computes a sequence of residues and can be represented as follows:

$$r_0 = a \tag{5a}$$
$$r_1 = b \tag{5b}$$
$$r_2 = r_0 - q_1 \cdot r_1 \tag{5c}$$
$$\ldots$$
$$r_{n+1} = r_n - q_{n-1} \cdot r_{n-1} \tag{5d}$$

We have prepended the equations (5a) and (5b) to make the presentation simpler. The equation (5c) corresponds to equation (1). q_n are the quotients. We claim that all the residues are a linear combination of a and b and prove it by induction. Consequently also the last non-zero residue, which is $gcd(a, b)$, is a linear combination of of a and b. First we have:

$$r_0 = a = a \cdot 1 + b \cdot 0 \tag{6a}$$
$$r_1 = b = a \cdot 0 + b \cdot 1 \tag{6b}$$

Suppose that

$$r_n = a \cdot x_n + b \cdot y_n \tag{6c}$$

for all $i = 0, 1, .. n$ and compute

$$\begin{aligned} r_{n+1} &= r_n - q_{n-1} \cdot r_{n-1} \\ &= (a \cdot x_n + b \cdot y_n) - q_{n-1} \cdot (a \cdot x_{n-1} + b \cdot y_{n-1}) \\ &= a \cdot (x_n - q_{n-1} \cdot x_{n-1}) + b \cdot (y_n - q_{n-1} \cdot y_{n-1}) \\ &= a \cdot x_{n+1} + b \cdot y_{n+1} \end{aligned} \tag{6d}$$

with

$$x_{n+1} = x_n - q_{n-1} \cdot x_{n-1} \tag{7a}$$
$$y_{n+1} = y_n - q_{n-1} \cdot y_{n-1} \tag{7b}$$

The above algorithm can immediately be transformed into a program:

```
def gcd_ext(a, b):
    x0, y0 = 1, 0
    x1, y1 = 0, 1
```

```
    while b:
        q = a // b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
        a, b = b, a % b
    return a, x0, y0
```

The program returns a triple g, x, y such that:

$$g = gcd(a, b) = a \cdot x + b \cdot y$$

Multiplicative Inverse (mod p)

In the public cryptography we need to find efficiently a number y that is the multiplicative inverse of a number $b \ (mod \ p)$, i.e.

$$b \cdot y \equiv 1 \ (mod \ p) \tag{8}$$

Let us mention that this equation does not always have a solution. E.g. when b and p are both even numbers then $b \cdot y \ (mod \ p)$ is even and can never produce 1. However, when b and p are relatively prime then we know from the Bézout's identity that

$$p \cdot x + b \cdot y = 1 \tag{9}$$

for some x and y and and hence y is the solution of equation (8). So, we can compute x using the Extended Euclidean Algorithm applied to p and b:

```
def inv(b, p):
    g, x, y = gcd_ext(p, b)
    return y % p
```

The last statement in this algorithm guarantees that

$$0 \leq y < p$$

Links:

- P. Keef and D. Guichard : Introduction to Higher Mathematics
- The Euclidean Algorithm and the Extended Euclidean Algorithm (DI Management Services)